



CRIPTOGRAFÍA BÁSICA

Diego Soria Ruiz y Gabriel de Miguel García



PARTE I

Representación de datos, codificaciones y cifrados clásicos

Criptografía - Representación de los datos

Es muy común encontrar los datos en diferentes formatos.

ASCII

Representa los caracteres como entradas de una tabla de números. A cada carácter le corresponde un valor de la tabla ASCII.

DEC	HEX	Simbolo	DEC	HEX	Simbolo
64	40h	@	96	60h	`
65	41h	A	97	61h	a
66	42h	B	98	62h	b
67	43h	C	99	63h	c
68	44h	D	100	64h	d
69	45h	E	101	65h	e
70	46h	F	102	66h	f
71	47h	G	103	67h	g
72	48h	H	104	68h	h
73	49h	I	105	69h	i
74	4Ah	J	106	6Ah	j
75	4Bh	K	107	6Bh	k
76	4Ch	L	108	6Ch	l
77	4Dh	M	109	6Dh	m
78	4Eh	N	110	6Eh	n
79	4Fh	O	111	6Fh	o
80	50h	P	112	70h	p
81	51h	Q	113	71h	q
82	52h	R	114	72h	r
83	53h	S	115	73h	s
84	54h	T	116	74h	t
85	55h	U	117	75h	u
86	56h	V	118	76h	v
87	57h	W	119	77h	w
88	58h	X	120	78h	x
89	59h	Y	121	79h	y
90	5Ah	Z	122	7Ah	z
91	5Bh	[	123	7Bh	{
92	5Ch	\	124	7Ch	
93	5Dh	]	125	7Dh	}
94	5Eh	^	126	7Eh	~
95	5Fh	_			

elCodigoASCII.com.ar

65 83 67 73 73

82 101 112 114 101 115 101 110 116 97 32 108 111
115 32 99 97 114 97 99 116 101 114 101 115 32 99
111 109 111 32 101 110 116 114 97 100 97 115 32
100 101 32 117 110 97 32 116 97 98 108 97 32 100
101 32 110 250 109 101 114 111 115 32 46 32 32 65
32 99 97 100 97 32 99 97 114 225 99 116 101 114 32
108 101 32 99 111 114 114 101 115 112 111 110 100
101 32 117 110 32 118 97 108 111 114 32 100 101
32 108 97 32 116 97 98 108 97 32 65 83 67 73 73 46

**01000010 01001001 01001110 01000001
01010010 01001001 01001111**

01010101 01110100 01101001 01101100 01101001 01111010 01100001 00100000
01100010 01100001 01110011 01100101 00100000 00110010 00100000 01100011
01101111 01101101 01101111 00100000 01110010 01100101 01110000 01110010
01100101 01110011 01100101 01101110 01110100 01100001 01100011 01101001
11110011 01101110 00100000 01100100 01100101 00100000 01101100 01101111
01110011 00100000 01100100 01100001 01110100 01101111 01110011 00101110
00100000 01010011 01101111 01101100 01101111 00100000 01110101 01110100
01101001 01101100 01101001 01111010 01100001 00100000 00110001 01110011
00100000 01111001 00100000 00110000 01110011 00101110 1000000001011

BINARIO

Utiliza base 2 como representación de los datos. Solo utiliza 1s y 0s.

Criptografía - Codificaciones y cifrados

La diferencia está en la clave: una codificación no tiene clave y cualquiera puede revertirla, así que **no es segura en la práctica**. Se pueden interpretar como una traducción.
Sin conocimiento de la clave (segura), el cifrado no podrá ser revertido y no dará pistas sobre el mensaje original.

Codificaciones

- Representan la misma información de otra forma. No usan clave.
- Cualquiera puede revertirla: no dan confidencialidad
- Algunos ejemplos son Base64, Base32 o ASCII
- Uso para compatibilidad de caracteres

Cifrados

- Ocultan la información mediante un algoritmo y una clave secreta.
- Reversible pero solo si conoces la clave
- Algunos ejemplos son ROT-N/César o Vigenère
- Sin la clave, hay que recurrir a la "fuerza bruta"

Criptografía - Las Bases

- Son un sistema de codificación matemático que usa un alfabeto específico.
- Una base N , utilizará N caracteres para su alfabeto. Comenzando por el 0.
- Conforme aumenta la base, utilizará más caracteres (numéricos $\rightarrow$ letras (Aa) $\rightarrow$ especiales (+, -, etc.)
- Representan la información a través de potencias y del módulo de N .
- Ejemplos cotidianos son el sistema decimal (Base 10), los relojes analógicos (Base 12) o el lenguaje binario (Base 2).

Base N :

$\frac{X_5}{\uparrow\uparrow\uparrow}$	$\frac{X_4}{\uparrow\uparrow\uparrow}$	$\frac{X_3}{\uparrow\uparrow\uparrow}$	$\frac{X_2}{\uparrow\uparrow\uparrow}$	$\frac{X_1}{\uparrow\uparrow\uparrow}$	$\frac{X_0}{\uparrow\uparrow\uparrow}$	$X \pmod{N}$
N^5	N^4	N^3	N^2	N^1	N^0	

Criptografía - Bases II

CODIFICADO

(De texto plano a Base N)

Dividimos por N y escribimos los restos en orden inverso, añadiendo también el último cociente).

Cada posición será un coeficiente de potencia de N.

DECODIFICADO

(De Base N a texto plano)

Debemos numerar el cifrado y otorgar a cada posición, una potencia de N (en ascendente desde N^0).

Multiplicamos cada coeficiente por su potencia y los sumamos.

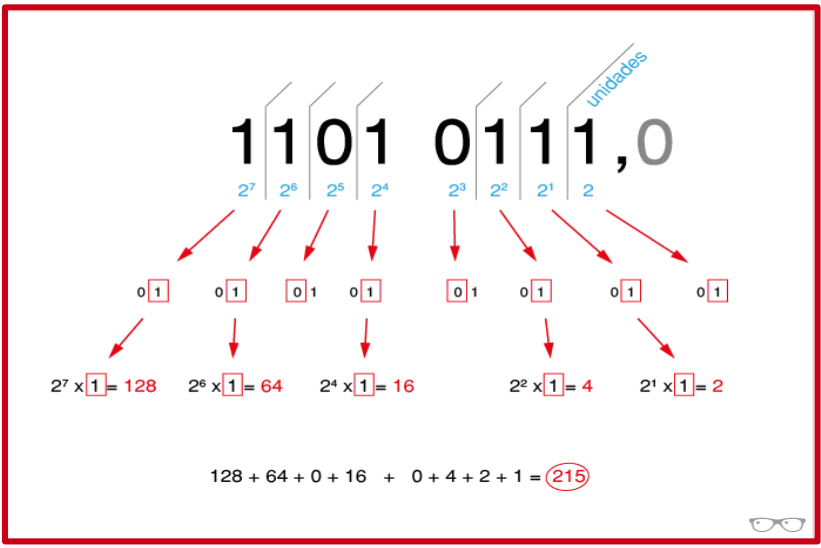
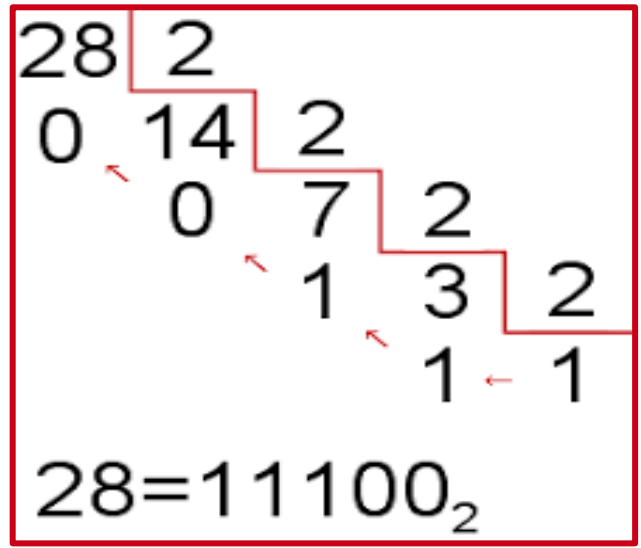
Nota: Para codificar/descodificar letras según una base, primero obtenemos su valor ASCII (G = 71) y operamos con dicho valor

BINARIO

Es Base2

Es una **codificación** que representa cualquier dato binario usando 2 caracteres imprimibles (1 y 0)

Se utiliza como lenguaje máquina y en electrónica gracias a su dualidad: 1 suele significar "encendido" o "le llega corriente" y 0 suele ser "apagado" o "ausencia de corriente"



Criptografía - Codificaciones

BASE64

Es una **codificación** que representa cualquier dato binario usando 64 caracteres imprimibles: A-Z, a-z, 0-9, + y /.

Cada 3 bytes se convierten en 4 caracteres. **Se identifica por su alfabeto** y porque su longitud es múltiplo de 4. Puede acabar en =, en == o sin relleno, según la longitud del texto original.

Texto original

CTF{Esto es un texto en Base64. También existen otras como Base32, Base58 o Base85, por ejemplo}

Texto en Base64

QIRGe0VzdG8gZXMgdW4gdG
V4dG8gZW4gQmFzZTY0LiBUY
WlIacOpbiBleGlzdGVuIG90cmF
zlGNvbW8gQmFzZTMyc0Y
XNINTggbyBCYXNlODUsIHBvc
iBlamVtcGxvfQ==

Criptografía - ¿Qué tengo delante?

¿Cómo sabemos si un texto está codificado a simple vista?. Pistas rápidas:

Formato	Pista para reconocerlo	"CTF{flag}" codificado
Hexadecimal	Solo 0-9 y a-f, longitud par	43 54 46 7b 66 6c 61 67 7d
Base32	Solo mayúsculas A-Z y 2-7, relleno con =	INKEM63GNRQWO7I=
Base64	A-Z a-z 0-9 + /, longitud múltiplo de 4	Q1RGe2ZsYWd9
Base58	Como Base64 pero sin 0, O, I, l, + ni /	rhzTmyGrnEYk
Base85	Casi cualquier símbolo imprimible, a veces entre <~ ~>	<~6W?O%Ao (mgI/~>
Decimal / binario	Números del 32 al 126 / grupos de 8 bits	67 84 70 123... / 01000011...

- CyberChef → operación **Magic**: prueba decodificaciones automáticamente y te sugiere la receta.
- Suele haber **varias capas**: si al decodificar sale otra cosa rara, identifica y repite.

Criptografía - Cifrados por Sustitución

Consisten en la sustitución letra por letra de un mensaje a partir de un alfabeto de igual longitud al usado por el mensaje cifrado y de una ordenación criptográfica de este.

En estos cifrados, la clave indica la ordenación del nuevo alfabeto cifrado.

No cambia el orden de los elementos del mensaje original, solo los sustituye.

Debido a su naturaleza, son fácilmente atacables mediante KPA (Known Plaintext Attack):

- Mediante un análisis de frecuencia, podemos llegar a encontrar un patrón que lo relacione con el alfabeto original del mensaje (por ejemplo, el castellano).
- Conociendo parte del mensaje y su contraparte en el alfabeto cifrado.
- Mediante combinaciones o palabras habituales de un determinado número de letras.

Letra	Frecuencia
E	13,7 %
A	12,5 %
O	8,7 %
S	8,0 %
R	6,9 %
N	6,7 %
I	6,3 %

- Herramientas: **quipqiup.com** (inglés) y **dCode** → **Substitution** (permite elegir castellano).

Criptografía - Codificaciones y cifrados

Cifrado de César



ROT-N

Cada letra se desplaza N posiciones en el alfabeto: N es la clave.

Al haber solo 26 posibles claves (en español), es muy fácil probar todas para encontrarla.

Está basado en el Cifrado Afín y su fórmula:

$$E(x) = Ax + B \pmod{m}$$

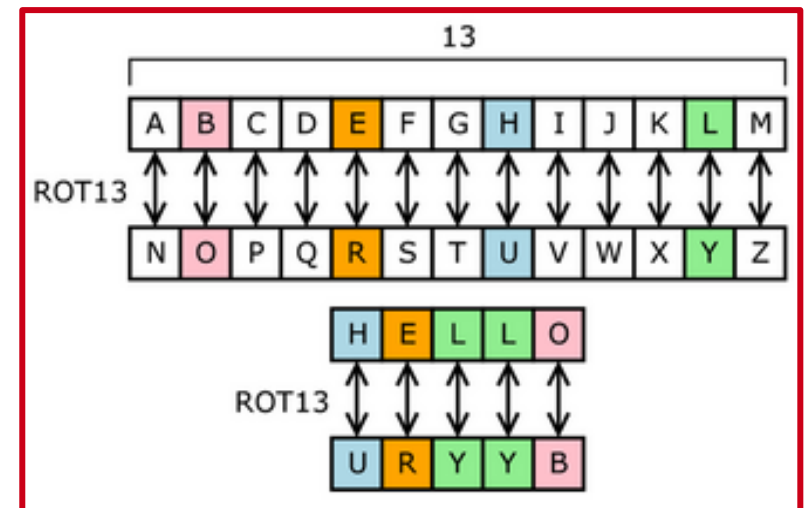
Con $A = 1$ y $B = N$

Texto original

CTF{El rot solo va a
modificar las letras, pero
no las llaves}

Texto en ROT 13

PGS{Ry ebg fbyb in n
zbqvsvpne ynf yrgenf, creb ab
ynf yynirf}



Criptografía - Tablas de Cifrado (Vigenère)



Vigenère

Se basa en una **tabla con dos entradas**. Una será la **clave** y la otra **el texto a cifrar**. Cada letra se desplaza según la letra de la clave que le toca, y la clave se repite.

Sin clave también se puede romper: se saca su longitud (Kasiski) y se ataca cada posición como un César.

<https://www.dcode.fr/cifrado-vigenere>

https://gchq.github.io/CyberChef/#recipe=Vigen%C3%A8re_Decompose

Criptografía - Tablas de Cifrado (Vigenère)

ENTRADA TEXTO PLANO		A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
ENTRADA CLAVE	A	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
	B	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
	C	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
	D	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
	E	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
	F	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
	G	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
	H	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
	I	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
	J	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
	K	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
	L	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
	M	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
	N	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
	O	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
	P	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
	Q	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
	R	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
	S	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
	T	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
	U	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
	V	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
	W	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
	X	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
	Y	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
	Z	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y

Texto original

CTF{Mi clave de cifrado es
Chachipiruli}



Clave = Chachipiruli



Texto en Vigenère

EAF{Op kaimy om epfthld
mj Wsiesoirpzjtz}

Interpretación por Tabla:

Relacionar fila (mensaje) con
columna (clave), el cifrado es
la coordenada resultante.

Interpretación

Matemática: Valores según
índice (A=0, B=1...), el
cifrado será la suma en
módulo 26.

Criptografía - Alfabetos gráficos

Pigpen (francomasónico)

Texto original

CTF{Hay cifrados de todo
tipo}

A	B	C	J	K	L
D	E	F	M	N	O
G	H	I	P	Q	R
S T U V			W X Y Z		

Texto en Pigpen

└ ⊠ □ ▯ ⊥ ⊐ └ ┐
□ ⊗ ⊥ ⊐ ⊗ ⊠ ⊐ □
⊠ ⊗ ⊐ ⊗ ⊠ ┐ ⊗ ⊗



DCode: <https://www.dcode.fr/chiffre-pig-pen-francs-macons>



Download CyberChef
Last build: 5 days ago
Options
About / Support

Operations
506

Search...

Favourites

Data format

Encryption / Encoding

Public Key

Arithmetic / Logic

Networking

Language

Utils

Date / Time

Extractors

Compression

Hashing

Code tidy

Forensics

Multimedia

Other

Recipe

From Binary

Delimiter
Space

Byte Length
8

From Base32

Alphabet
A-Z2-7=

☐ Remove non-alphabet chars

From Hex

Delimiter
Space

From Base64

Alphabet
A-Za-z0-9+/=

☒ Remove non-alphabet chars
☐ Strict mode

STEP
BAKE!
Auto Bake

Input

01000101 01001001 01000010 01010101 01000111 00110100 01010001 01000100 01001001 01011001 01011010 01000001
01000111 01010001 01011010 01010011 01000001 01001110 01000010 01010011 01000101 01000001 00110011 01000100
01010011 01001001 01000010 01010111 01000111 01000101 01010001 01000100 01001011 01001110 01011010 01000001
01000111 01010101 00110011 01000011 01000001 01001110 01011010 01010110 01000101 01000001 00110011 01000100
01001001 01001001 01000010 01010111 01001101 01010001 01010001 01000100 01001011 01001110 01010010 01000001
01000111 00110100 00110010 01010011 01000001 01001110 01010010 01010010 01000101 01000001 00110010 01010100
01001111 01001001 01000010 01010110 01000111 01001001 01010001 01000100 01001111 01001110 01010010 01000001
01000111 01011001 01011010 01010011 01000001 01001110 01011010 01011010 01000101 01000001 00110010 01000100
01000101 01001001 01000010 01010111 01000001 01001110 01010001 01000100 01000101 01001111 01001010 01000001
01000111 01010001 00110010 01010011 01000001 01001110 01000100 01000110 01000101 01000001 00110011 01010100
01010011 01001001 01000010 01010111 01000111 01000101 01010001 01000100 01001011 01001111 01000010 01000001
01000111 01010001 01011010 01000011 01000001 01001101 01011010 01010001 01000101 01000001 00110011 01000100
01000101 01001001 01000010 01011000 01001000 01000101 01010001 01000100 01001001 01001101 01001010 01000001
01000111 01001101 00110011 01000011 01000001 01001110 01001010 01010011 01000101 01000001 00110010 01000100
01000011 01001001 01000010 01010100 01001101 01010001 01010001 01000100 01000111 01011010 01000001 00111101

Output

Hola gente del CTF, bienvenidos a Cripto :d





Buscar una herramienta

★ [BUSCAR EN dCODE](#)
 Por ejemplo, escriba 'cesar'

★ [EXPLORE LA LISTA COMPLETA DE HERRAMIENTAS DE dCODE](#)

Resultados









1	2	3	4	5	
1	A	B	C	D	E
2	F	G	H	I	J
3	K	L	M	N	O
4	P	Q	R	S	T
5	U	V	W	X	Y

331522514445113432114422354524131144

Cifra de Polibio - [dCode](#)

Etiqueta(s): Cifrado de Sustitución, Cifrado con cuadrícula

Compartir







dCode y más

dCode es gratuito y sus herramientas son una valiosa ayuda en juegos, matemáticas, geocaching, acertijos y problemas para resolver todos los días.

¿Una sugerencia? ¿Un comentario? ¿Un error? ¿Una idea? ¡Escribe en dCode!

CIFRA DE POLIBIO

Criptografía > Cifrado de Sustitución > Cifra de Polibio

DESCIFRANDO CON POLIBIO

★ MENSAJE CIFRADO POR POLIBIO

★ LENGUAJE DE TEXTO (ASUMIDO) Español

☐ USAR CUADRÍCULA PREDETERMINADA (5x5 SIN TECLA -J)
☐ INTENTAR DESCIFRADO AUTOMÁTICO
☐ PROBANDO CUADRÍCULAS COMUNES (ATAQUE DE DICCIONARIO)
☐ TRANSFORMAR EN SUSTITUCIÓN MONOALFABÉTICA
☒ CONOZCO LA CUADRÍCULA O CLAVE DE CIFRADO

\	1	2	3	4	5
1	A	B	C	D	E
2	F	G	H	I	J
3	K	L	M	N	O
4	P	Q	R	S	T
5	U	V	W	X	Y

5

5

RESIZE

CLEAR

L ABCDEFGHIJKLMNPOQRSTUVWXYZ

★ ORDEN DE COORDENADAS ☒ FILA, COLUMNA (CLÁSICO)
☐ COLUMNA, FILA (RARO)

[▶ DESCIFRAR](#)

Ver también : [Nihilist Cipher](#) – [Tap Code Cipher](#)

CIFRADO CON POLYBIUS

★ TEXTO PLANO PARA ESCRIBIR CON POLYBIO

★ CLAVE DE CUADRÍCULA O DE CIFRADO

\	1	2	3	4	5
1	A	B	C	D	E
2	F	G	H	I	J

5

5

RESIZE

Resumen

- ★ Descifrando con Polibio
- ★ Cifrado con Polybius
- ★ ¿Qué es el cifrado de Polibio? (Definición)
- ★ ¿Cómo codificar con Polybio?
- ★ ¿Cómo generar la cuadrícula de Polibio?
- ★ ¿Cómo descifrar por Polibio?
- ★ ¿Cómo reconocer la cifra de Polibio?
- ★ ¿Cómo descifrar a Polibio sin la cuadrícula?
- ★ ¿Cuáles son las variantes del cifrado de Polibio?
- ★ ¿Cuándo se inventó el cifrado de Polibio?

Páginas similares

- ★ Tap Code Cipher (English)
- ★ Nihilist Cipher (English)
- ★ Bazeries Cipher (English)
- ★ Grid Coordinates (English)
- ★ Grandpré Cipher (English)
- ★ Bifid Cipher (English)
- ★ Collon Cipher (English)
- ★ LISTA DE HERRAMIENTAS DE dCODE

Apoyar dCode

Participate in the Discord forum:

- ★ [Discord](#)

Foro/Ayuda

(Solo en ingles)



```
echo "dGV4dG8gZW4gYmFzZTY0" | base64 -d
```

```
cat base64.txt | base64 -d
```

```
base64 -d base64.txt
```

```
echo "486f6c61" | xxd -r -p
```

```
xxd -r -p hexa.txt
```




PARTE 2

Operación XOR y hashes

XOR

XOR

- Cifrado que opera XOR en binario
- Utiliza una **clave secreta**.
- Como la **longitud** de la **clave** suele ser **menor al texto**, se repetirá **cíclicamente**.

Propiedades

1. Conmutativa: $A \text{ xor } B = B \text{ xor } A$

2. Asociativa: $(A \text{ xor } B) \text{ xor } C = A \text{ xor } (B \text{ xor } C)$

3. Autoinversa: $(A \text{ xor } B) \text{ xor } B = A$

<i>A</i>	<i>B</i>	XOR
0	0	0
0	1	1
1	0	1
1	1	0

XOR

ATAQUE DE TEXTO CLARO CONOCIDO DADO UN PAR CLARO/CIFRADO

Aplicando la propiedad autoinversa del XOR podremos recuperar la clave

$$(A \text{ xor } B) \text{ xor } B = A$$

Texto en claro: A

Texto cifrado: $(A \text{ xor } K)$



Clave: $\text{Texto} \text{ xor } \text{Cifrado}$

Clave: $A \text{ xor } (A \text{ xor } K) = K$

XOR

EJEMPLO I

Esta vez nos dan directamente la flag, pero parece que está cifrada:

Flag: 13 3e 2b 24 3d 3d 14 02 66 1f 04 47 34 09 11 0e 32 0d 41 0b 27 4c 02 0b 27 1a 04 47 28
03 41 02 35 4c 0c 12 3f 4c 12 02 21 19 13 08 3b

Formato de la flag: URJC{ }

XOR

¿Cómo lo resolvemos?

Conocemos el formato de la flag...

CIFRADO	FLAG
13	U
3e	R
2b	J
24	C
3d	{
3d	?
14	?
02	?
66	?
...	...

$$(A \text{ xor } B) \text{ xor } B = A$$

Tenemos:

Cifrado = Texto[i] XOR Clave[i]

Flag = Texto[i]

Entonces:

Cifrado[i] XOR Texto[i] = Clave[i]

XOR

¿Cómo lo resolvemos?

CIFRADO		FLAG		CLAVE
13		U		k[0]
3e		R		k[1]
2b		J		k[2]
24		C		k[3]
3d	XOR	{	=	k[4]
3d		?		?
14		?		?
02		?		?
66		?		?
...		...		...

$$(A \text{ xor } B) \text{ xor } B = A$$

Tenemos:

Cifrado = Texto[i] XOR Clave[i]

Flag = Texto[i]

Entonces:

Cifrado[i] XOR Texto[i] = Clave[i]

XOR

EJEMPLO II

¡Ayúdanos a descifrar este texto! Conocemos la correspondencia de algunas cadenas:

"cifrado muy utilizado" = 0c 2d 03 3e 00 31 3d 6a 2e 36 2d 66 16 1b 04 1c 0c 0e 08 10 06

"propiedades importantes" = 3c 13 3a 22 23 26 27 35 22 06 1c 4d 19 08 04 06 06 1d 17 01 30 00 3f

06 38 66 3b 20 3f 50 00 07 49 01 07 56 0c 2d 03 3e 00 31 3d 6a 2e 36 2d 66 16 1b 04 1c 0c
0e 08 10 06 56 0a 2a 45 20 00 75 31 38 2a 33 20 29 04 1d 0c 16 0c 15 63 20 00 13 01 21 45 3c
13 3a 22 23 26 27 35 22 06 1c 4d 19 08 04 06 06 1d 17 01 30 00 3f 6b 16 27 2b 2d 27 3b 66 17
06 08 1e 00 07 49 01 07 56 0c 2d 03 3e 00 31 3d 6a 1b 0c 06 66 1a 4f 0e 1f 0b 1b 0a 11 1a 56 1f
25 17 38 04 75 36 2f 2f 63 20 23 1b 1b 02 50 00 1a 49 17 05 17 1d 2b 45 3c 0e 31 20 ab 30 63
35 36 0f 06 0e 11 17 54 05 15 1a 56 1f 36 0a 3c 08 30 36 2b 27 26 27 66 07 0a 01 50 3d 3b 3b
54 19 17 1d 25 45 3f 00 36 33 38 63 2f 35 66 00 03 0c 06 00 58 49 54 0d 13 1c 27 0c 2a 13 34 20
26 2c 63 2d 66 00 00 03 03 00 13 1c 1d 1b 56 03 25 45 2a 0d 34 35 40 16 11 1e 05 18 37 22 22
45 11 1a 54 0f 17 0c 2d 09 31

XOR

EJEMPLO III: Alice y Bob quieren intercambiar mensajes

¡Te mando la contraseña secreta cifrada con mi clave!

16 3e 2b 35 1e 06 11 0a 1e 0c 0e 00 43 63 08 0e 05 45 25 00 00 17 16 54 0d 50 63 0f 0d 17 13 36 45
0b 13 06 11 41 40 36 09 41 05 00 73 06 02 1c 06 11 0d 54 3e



Te la mando de vuelta cifrada con mi clave

5a 5f 78 50 79 73 7f 6e 7f 47 6b 79 0f 02 5b 6b 62 30 4b 64 61 5c 73 2d 41 31 30 6a 6a 62 7d 52 24 40
76 7f 5d 20 13 53 6e 34 6b 64 12 4d 67 65 4a 70 5e 31 59



Genial, voy a volver a cifrar otra vez con mi clave

19 33 19 26 1c 20 1a 0d 0d 22 1f 18 3e 41 37 0a 14 55 18 01 02 2e 16 59 20 00 73 06 0b 14 18 01 41
23 04 1a 29 41 22 10 02 55 1d 01 41 28 04 17 2f 04 3f 00 1a



Bob



Alice

XOR

¿Qué está pasando?



Bob



K1

$M1 = \text{Texto XOR } K1$



$M2 = M1 \text{ XOR } K2$



$M3 = M2 \text{ XOR } K1$



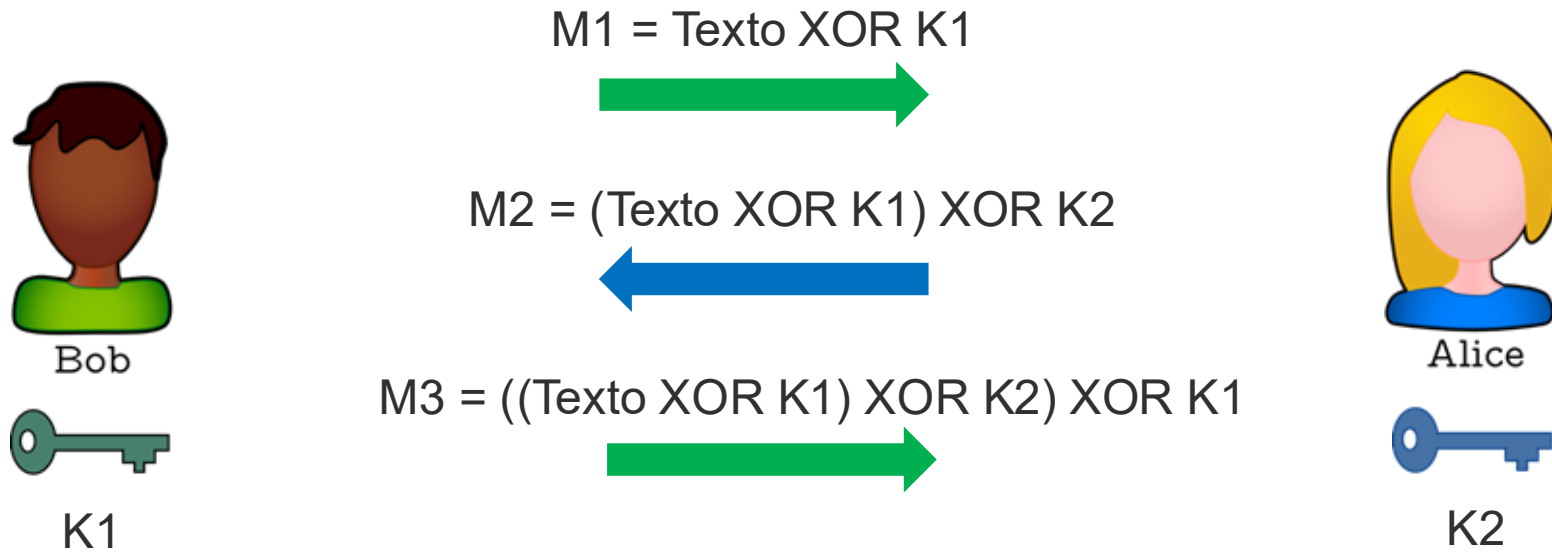
Alice



K2

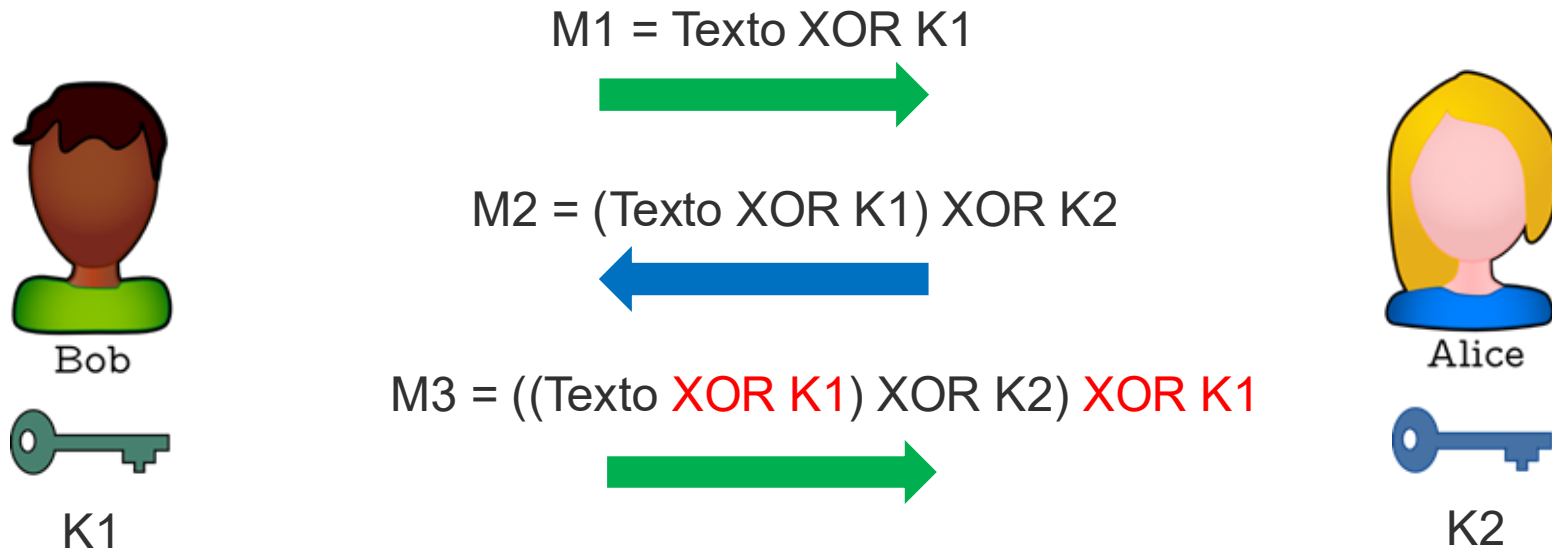
XOR

¿Qué está pasando?



XOR

¿Qué está pasando?



XOR - Automatizando con Python

CyberChef sirve para empezar, pero en cuanto el reto se complica conviene hacer un script:

```
cifrado = bytes.fromhex("133e2b243d3d1402...")    # Ejemplo I
conocido = b"URJC{"

clave = bytes(c ^ p for c, p in zip(cifrado, conocido))
print(clave)    # b"FlagF" -> la clave es b"Flag" y se repite

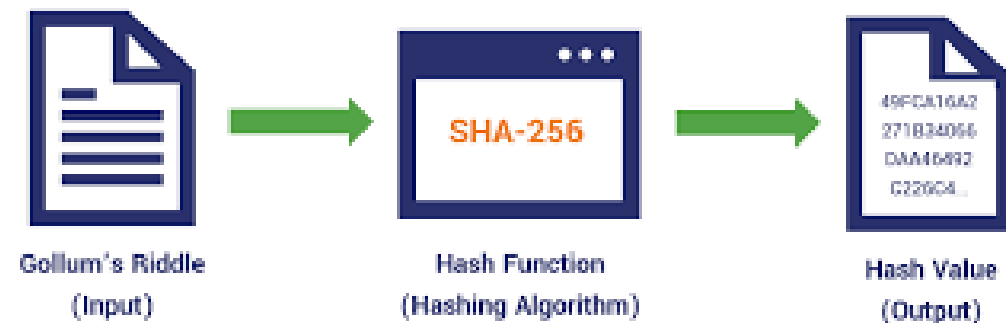
clave = b"Flag"
flag = bytes(c ^ clave[i % len(clave)] for i, c in enumerate(cifrado))
print(flag)
```

- Con pwntools es una línea: `from pwn import xor; xor(cifrado, b"Flag")`
- Si no conoces nada del texto: **xortool** estima la longitud de la clave y la clave.

¿Qué es un hash?

- Es una **función matemática o criptográfica**, resume la información
- Da como **resultado** una cadena de caracteres de longitud fija (**digest**), **independientemente** de la longitud entrada
- Es **irreversible**. Una vez aplicada **no se puede obtener el valor inicial**.

How Hashing Works



Propiedades

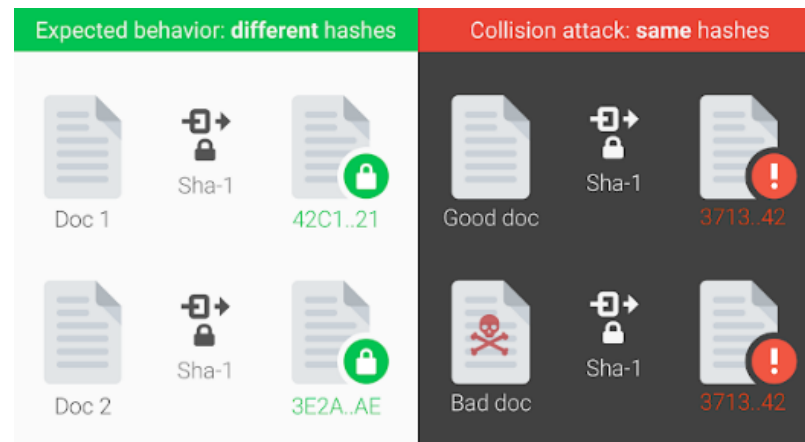
- **Collision Resistance (CR)**
 - Debe ser inviable encontrar dos mensajes cualesquiera $m \neq m'$ tal que $H(m) = H(m')$
- **Second preimage o target-collision resistance (TCR)**
 - Dado un mensaje m concreto, debe ser inviable encontrar otro $m' \neq m$ tal que $H(m) = H(m')$
- **Preimage resistance (PR)**
 - Dado un resumen h , debe ser inviable encontrar m tal que $H(m) = h$ (no se puede “dar la vuelta”)

Criptografía - Hashes

- Lo que sí puede hacerse es **pre-computar** los hashes de contraseñas típicas, porque el hash es determinista (misma entrada, mismo resultado). Así funcionan las rainbow tables y webs como CrackStation. Defensa: la sal (salt), un valor aleatorio por contraseña que inutiliza esas tablas
- **Conociendo la función utilizada** podemos realizar ataques de **fuerza bruta** sobre los hashes, de forma que, si en nuestro **diccionario** se encuentra la palabra *hasheada*, **sabremos qué esconde el hash**
- Es importante destacar que esto **NO ES LO MISMO QUE REVERTIR EL CÁLCULO**
- **Intentar adivinar un hash** por fuerza bruta pura se vuelve **muy costoso al crecer la longitud y el alfabeto, y más aún con hashes lentos como bcrypt o Argon2 (frente a MD5 o SHA-1, rapidísimos)**

Criptografía - Hashes

- Existen determinadas funciones hash cuyo uso **no se recomienda**
 - **MD5**
 - **SHA1**
- En ambos ya hay **colisiones reales (SHAttered, 2017)**

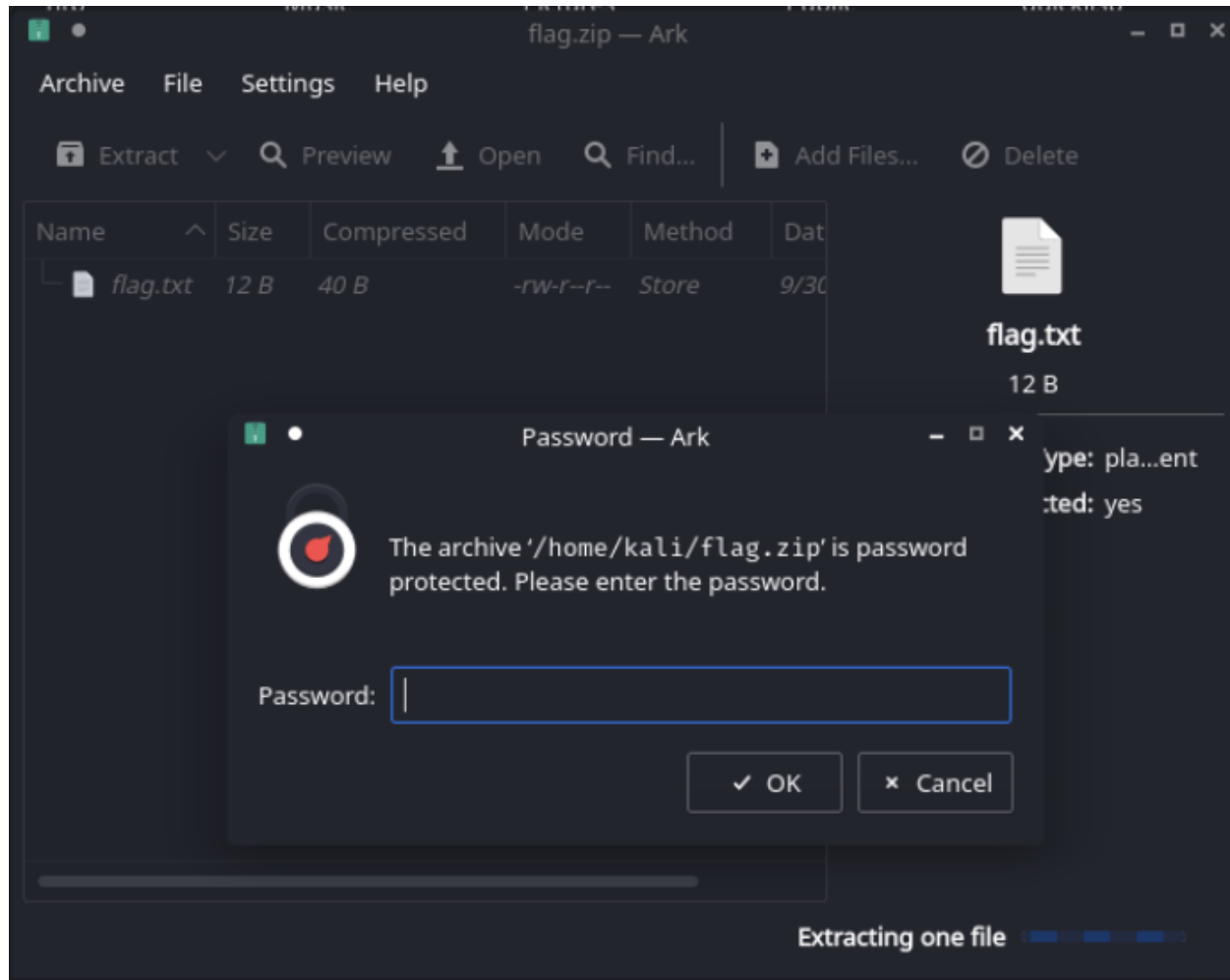


Criptografía - Hashes

- Cada **fichero** se puede resumir con un **valor hash**
- Existen herramientas que, dada una lista de **hashes**, nos automatizan el proceso de obtener un valor que genere dicho hash.
- **Así se sacan contraseñas de ficheros protegidos (zip2john + diccionario)**



Criptografía – Hashes (Ejemplo)



Criptografía – Hashes (Ejemplo)



```
~ : zsh — Konsole
File Edit View Bookmarks Plugins Settings Help
New Tab Split View Left/Right Split View Top/Bottom
(kali@kali)-[~]
$ zip2john flag.zip > hashZip
```

```
~ : zsh — Konsole
File Edit View Bookmarks Plugins Settings Help
New Tab Split View Left/Right Split View Top/Bottom Load a new tab with layout 2x2 terminals
(kali@kali)-[~]
$ cat hashZip | grep -E -o '(\$pkzip2\$.*\$/pkzip2\$)|(\$zip2\$.*\$/zip2\$)' > zipHash2hashcat
```

Criptografía – Hashes (Ejemplo)



```
~: zsh — Konsole
File Edit View Bookmarks Plugins Settings Help

New Tab Split View Left/Right Split View Top/Bottom

(kali@kali)-[~]
$ john hashZip --wordlist=wordlists.txt
Using default input encoding: UTF-8
Loaded 1 password hash (ZIP, WinZip [PBKDF2-SHA1 128/128 SSE2 4x])
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
Warning: Only 1 candidate left, minimum 16 needed for performance.
hola1234 (flag.zip/flag.txt)
1g 0:00:00:00 DONE (2021-09-30 16:55) 100.0g/s 100.0p/s 100.0c/s 100.0C/s hola1234
Use the "--show" option to display all of the cracked passwords reliably
Session completed

(kali@kali)-[~]
$ john hashZip --show
flag.zip/flag.txt:hola1234:flag.txt:flag.zip:flag.zip

1 password hash cracked, 0 left

(kali@kali)-[~]
$
```

```
(kali@kali)-[~]
$ hashcat -m 13600 zipHash2hashcat ./wordlists.txt
hashcat (v6.1.1) starting... using salt with
```

```
Session.....: hashcat
Status.....: Cracked
Hash.Name.....: WinZip
Hash.Target.....: $zip2$*0*3*0*f819c01513f1f5018f4e73128d711b52*8d6c* ... /zip2$
Time.Started.....: Thu Sep 30 16:59:35 2021 (0 secs)
Time.Estimated...: Thu Sep 30 16:59:35 2021 (0 secs)
Guess.Base.....: File (./wordlists.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 3 H/s (1.66ms) @ Accel:64 Loops:999 Thr:1 Vec:4
Recovered.....: 1/1 (100.00%) Digests
Progress.....: 1/1 (100.00%)
Rejected.....: 0/1 (0.00%)
Restore.Point....: 0/1 (0.00%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:0-999
Candidates.#1....: hola1234 → hola1234

Started: Thu Sep 30 16:58:55 2021
Stopped: Thu Sep 30 16:59:37 2021

(kali@kali)-[~]
$ hashcat -m 13600 zipHash2hashcat --show
$zip2$*0*3*0*f819c01513f1f5018f4e73128d711b52*8d6c*c*327662bd488eec34fe3ad3fa*4b36073395bdba927dda*$/zip2$:hola1234

(kali@kali)-[~]
$
```

Criptografía - Hashes: chuleta

1. Identificar el tipo: `hashid` o `name-that-hash`. 32 hex → MD5/NTLM · 40 → SHA-1 · 64 → SHA-256 · \$2b\$ → `bcrypt`
2. Probar primero en **CrackStation** (hashes comunes sin sal).
3. Si no sale, diccionario con John o hashcat:

```
zip2john flag.zip > hash
john hash --wordlist=/usr/share/wordlists/rockyou.txt
john hash --show

hashcat -m 0 hash.txt rockyou.txt      # modo según el tipo de hash
# 0 MD5 · 100 SHA-1 · 1400 SHA-256 · 1000 NTLM · 3200 bcrypt
# ZIP: 13600 (AES) · 17200-17230 (PKZIP clásico)
```

Criptografía - Codificar, cifrar y hashear

Tres cosas que se confunden mucho:

	Codificación	Cifrado	Hash
¿Usa clave?	No	Sí	No
¿Reversible?	Sí, por cualquiera	Sí, con la clave	No
Salida	Depende de la entrada	Depende de la entrada	Longitud fija
¿Para qué?	Representar o transportar datos	Confidencialidad	Integridad, guardar contraseñas
Ejemplos	Base64, hex, URL encoding	César, Vigenère, XOR, AES, RSA	MD5, SHA-256, bcrypt

Para practicar lo aprendido

- Para **practicar lo que hemos visto hasta ahora**, podéis realizar los **retos** de la categoría **Básica** de la plataforma **Atenea**

<https://atenea.ccn-cert.cni.es/challenges>

- Para seguir: CryptoHack (cryptohack.org), muy bueno para aprender **cripto moderna (XOR, RSA, AES) paso a paso**
- Y picoCTF (picoctf.org), con retos de **todas las categorías y dificultad progresiva**



SORPRESA FINAL

¿¿¿Qué es enigma???



Enigma

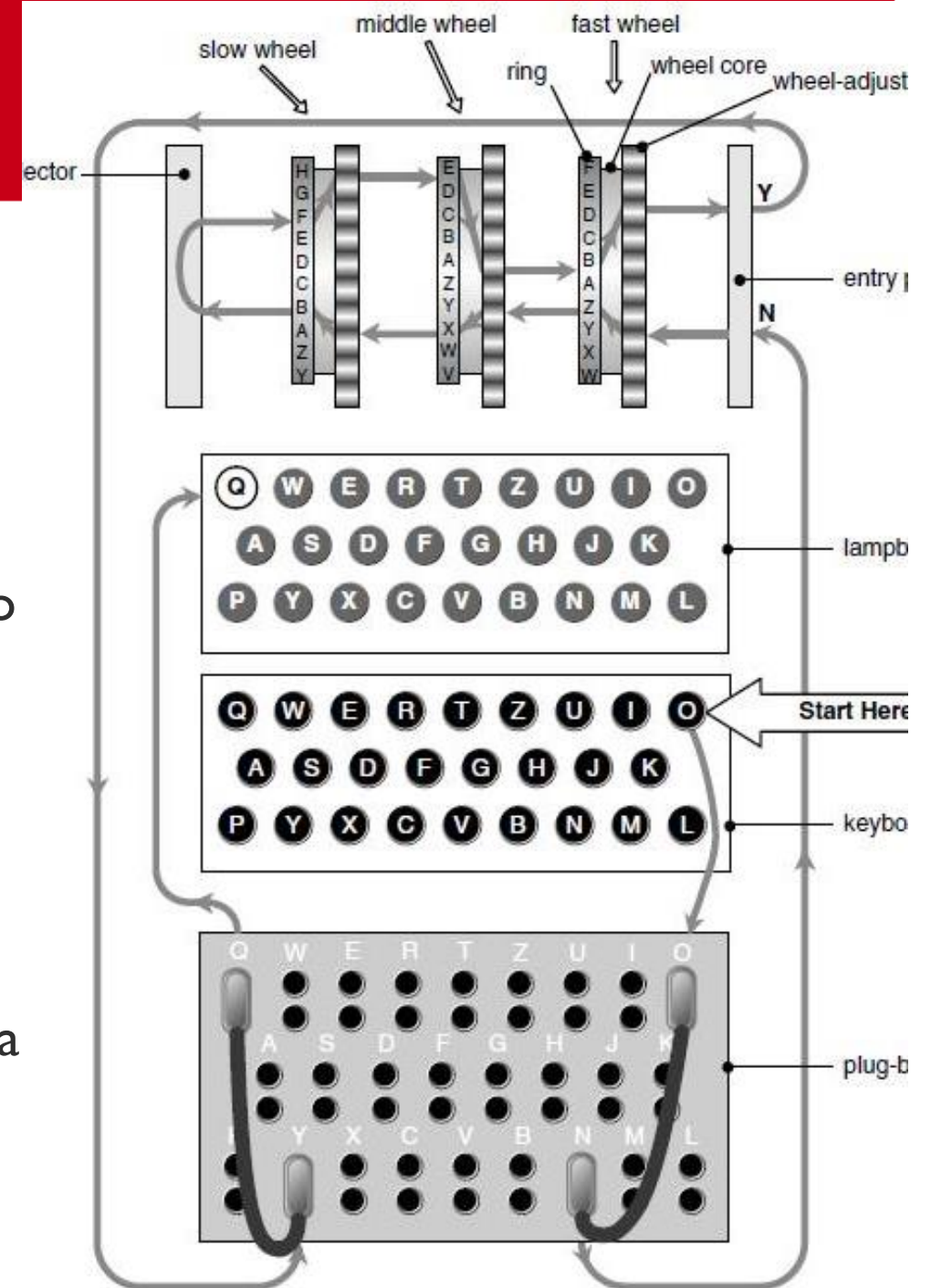


¿Qué es Enigma?

- Máquina de cifrado electromecánica usada por la Alemania nazi en la Segunda Guerra Mundial (años 30 y 40).
- Cifraba cada mensaje letra a letra: al pulsar una tecla, se encendía una bombilla con la letra cifrada.
- Su fuerza estaba en que la clave cambiaba en cada pulsación: la misma letra casi nunca se cifraba igual dos veces seguidas.
- Los alemanes la creían indescifrable. El número de configuraciones posibles superaba los 150 trillones.

Enigma

- Rotores: 3 discos giratorios que reordenaban el alfabeto. Cada tecla hacía girar al menos uno, cambiando el cableado.
- Reflector: devolvía la corriente por otro camino. Efecto secundario clave: hacía el cifrado reversible... pero introdujo su gran fallo.
- Clavijero (Steckerbrett): intercambiaba pares de letras antes y después, multiplicando las combinaciones.
- La CONFIGURACIÓN diaria (qué rotors, en qué posición y qué clavijas) era la clave secreta. Se cambiaba cada día a medianoche



Enigma



- Por culpa del reflector, una letra NUNCA podía cifrarse como ella misma. La A jamás salía como A.
- Parece un detalle inofensivo... pero redujo drásticamente las combinaciones que había que probar.
- En Bletchley Park (Reino Unido), un equipo liderado por Alan Turing construyó la "Bombe", una máquina que aprovechaba ese fallo y los mensajes predecibles (partes conocidas del texto).
- No rompían la máquina: reconstruían la CONFIGURACIÓN del día. Con ella, descifraban todo el tráfico.
- Con ello se consiguió que la guerra acabara antes y que nacieran las nuevas máquinas de cómputo.



RESOLVAMOS ENIGMA
